

Application Maintenance Manual

The Job Journal

Version 1.0
May 12, 2025

The Workforce Center of Blackbriar
Blackbriar City
Amber Kansfield

Table of Contents

Section 1. Overview	3
1.1 Purpose	3
1.2 Audience	3
Section 2. Back End Deployment	3
2.1 Commit Changes	3
2.2 Confirm Changes	4
2.3 Heroku Deployment	5
Section 3. Front End Deployment.....	6
3.1 Build	7
3.2 Deploy Changes.....	7
Section 4. Database	9
4.1 Connect to Database	9
4.2 Deletion.....	10
Section 5. GitHub Reset	10

Section 1. Overview

1.1 Purpose

The purpose of this document is to catalog the steps required to update the *Job Journal* application. This document will provide all the necessary information required to make changes and allow for a smooth transition between software versions.

1.2 Audience

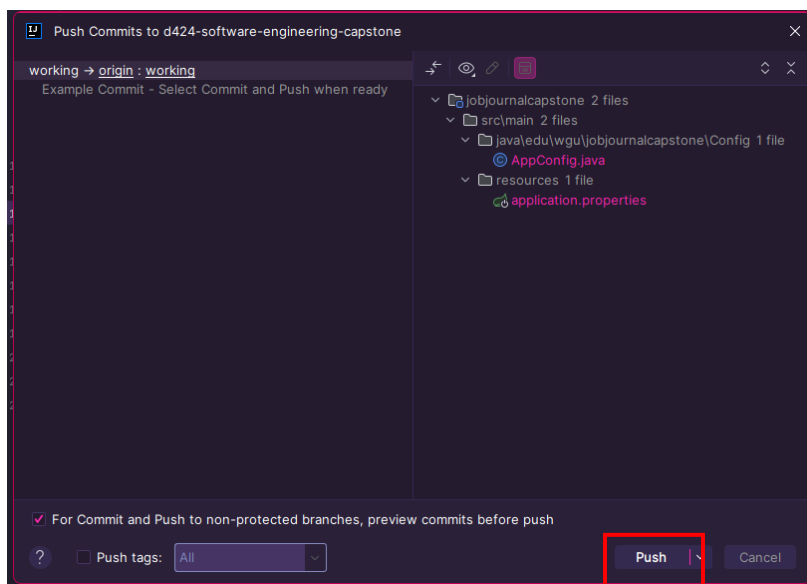
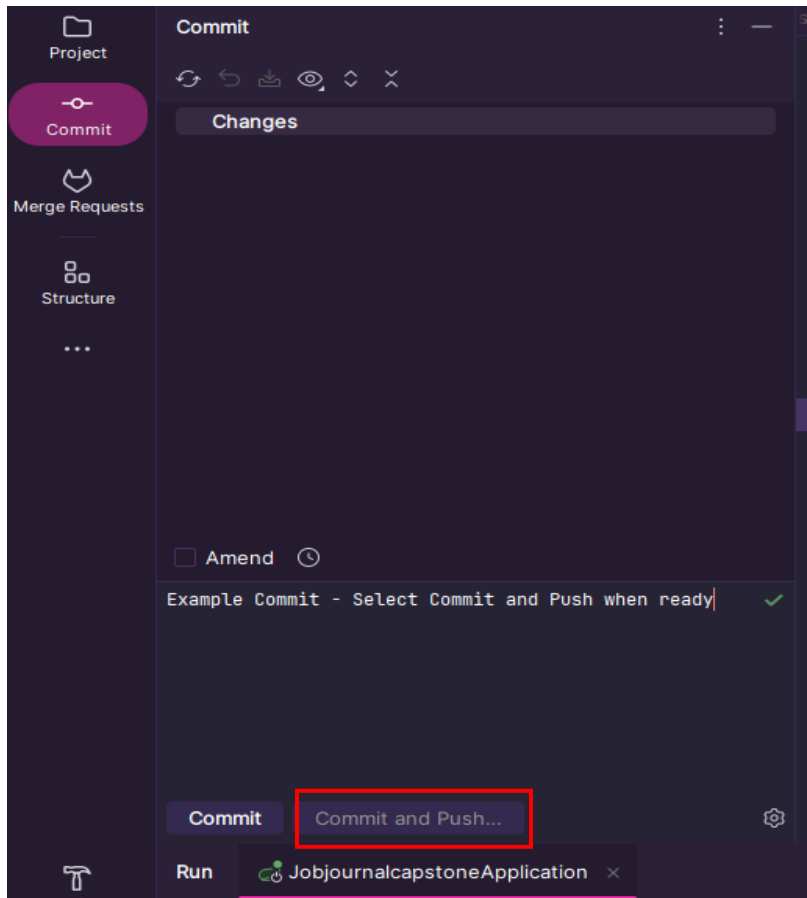
This document is intended for the maintenance team at The Workforce Center of Blackbriar City. The maintenance team will have full responsibility over application updates and maintenance after the agreed-upon hand-off date. The team should be familiar with both GitHub Pages and Heroku in order to ensure a smooth update experience.

Section 2. Back End Deployment

This section covers the deployment of the back-end Spring Boot server application to Heroku.

2.1 Commit Changes

Any changes made should be pushed to GitHub, either through the console or through IntelliJ's integrated git GUI.



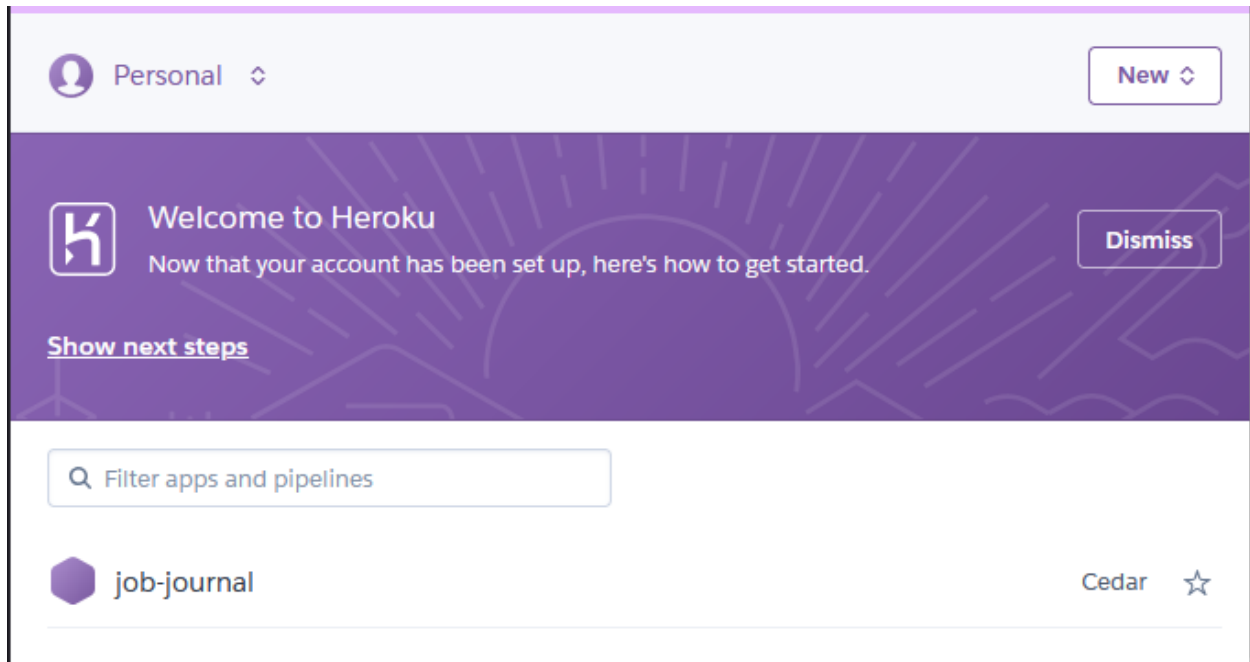
2.2 Confirm Changes

GitHub Link: <https://github.com/a-kansfield/JobJournal>

Check that the saved commit is appearing on GitHub and not the origin: GitLab.

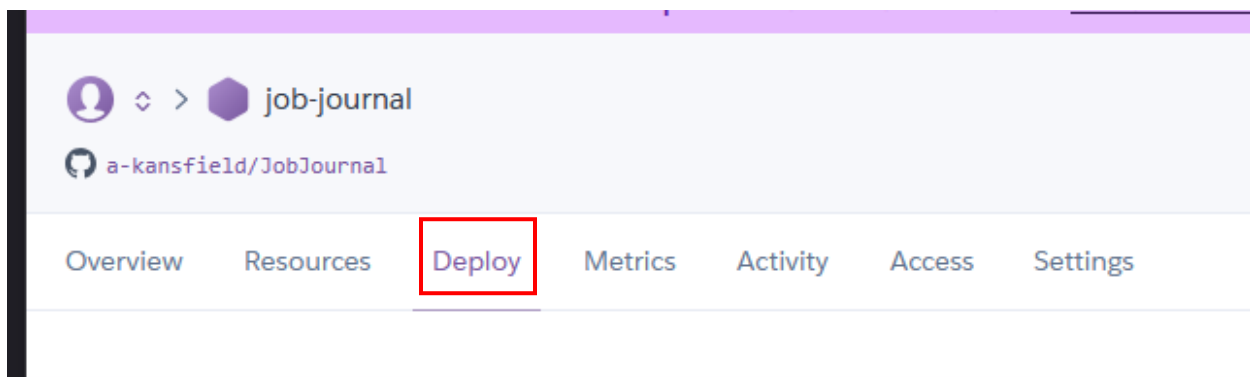
2.3 Heroku Deployment

A. Once changes have been confirmed, the next step is to log into Heroku (<https://www.heroku.com/>) with the provided credentials.



B. Select “job-journal”

C. Navigate to “Deploy.”



D. Deployment method should be set to GitHub, and the connected app should be *a-kansfield/JobJournal*.

Deployment method

Heroku Git
Use Heroku CLI

GitHub
Connected

Container Registry
Use Heroku CLI

App connected to GitHub

Code diffs, manual and auto deploys are available for this app.

Connected to [a-kansfield/JobJournal](#) by [a-kansfield](#) [Disconnect...](#)

Releases in the [activity feed](#) link to GitHub to view commit diffs

E. Lastly, under Manual Deploy, select “working” and press the “Deploy Branch” button.

Manual deploy

Deploy the current state of a branch to this app.

Deploy a GitHub branch

This will deploy the current state of the branch you specify below. [Learn more.](#)

Choose a branch to deploy

[Deploy Branch](#)

F. If the process succeeds, a message should appear stating that the app was successfully deployed, as shown below.

Receive code from GitHub

Build **working** `bc64f4cf`

Release phase

Deploy to Heroku

Your app was successfully deployed.

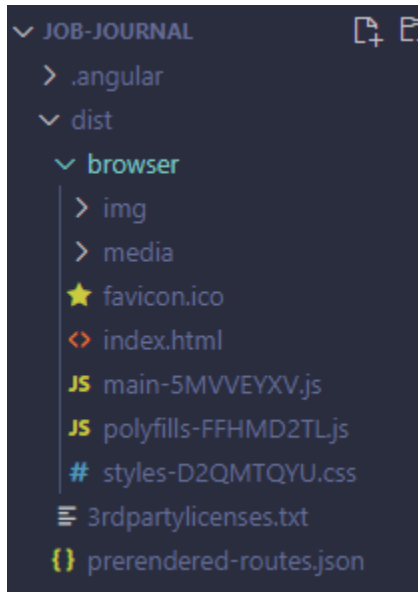
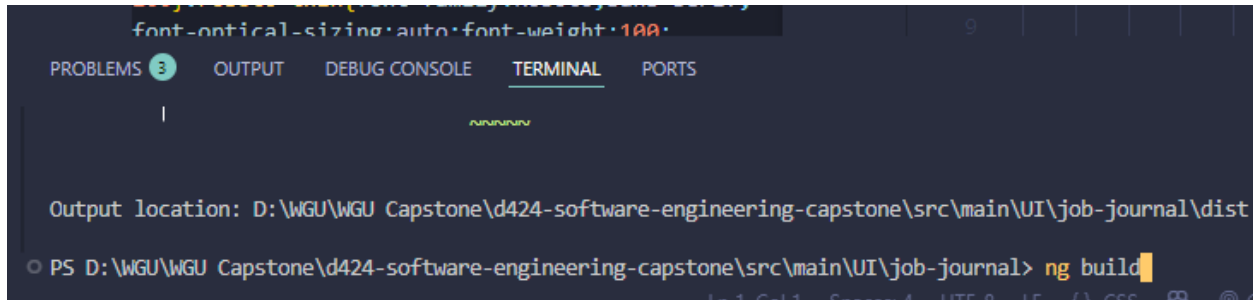
[View](#)

Section 3. Front End Deployment

This section covers the deployment of the front-end Angular application to GitHub Pages.

3.1 Build

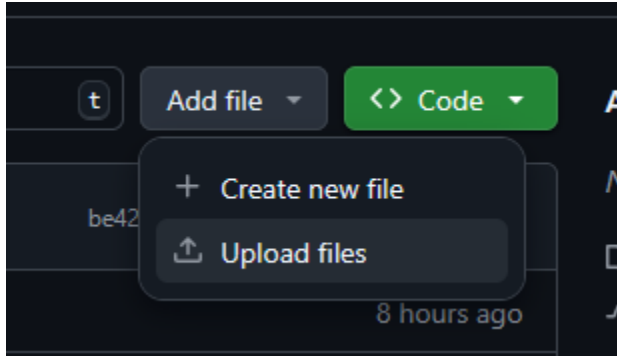
A. Once changes have been completed and tested thoroughly, build the app via command line with the command “ng build.” This will update the files in “dist/browser” within the Angular file structure.



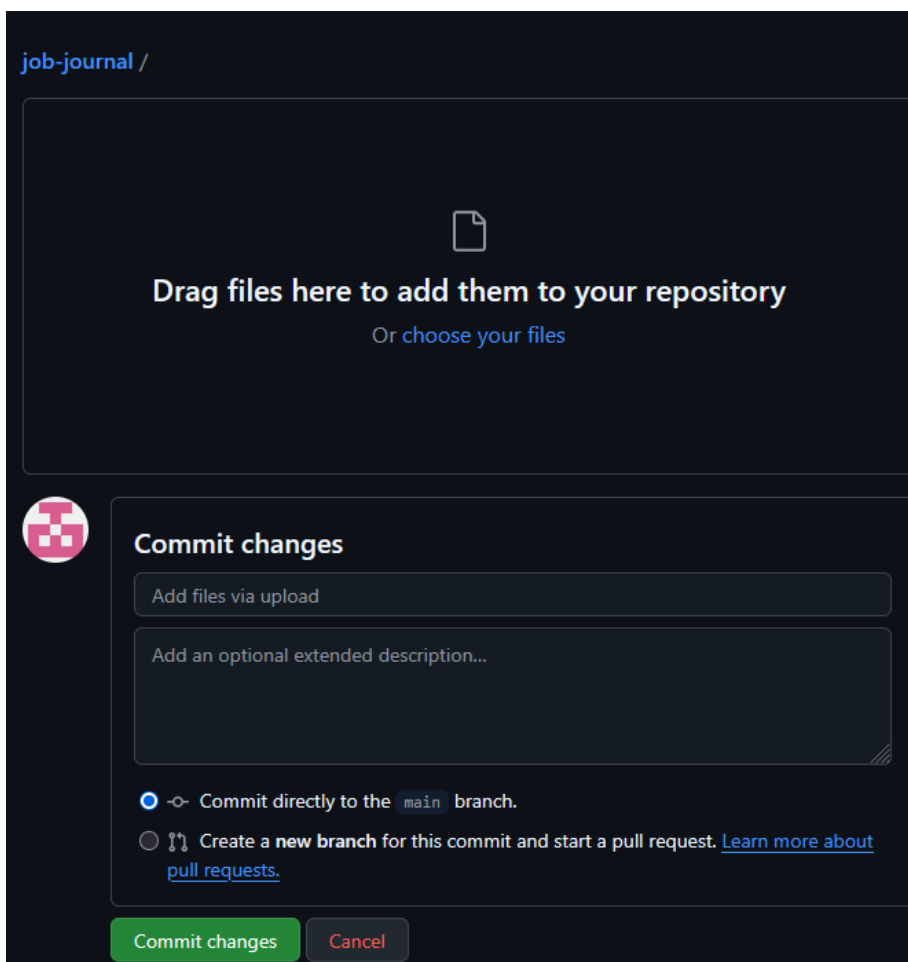
3.2 Deploy Changes

GitHub Link (Front-End): <https://github.com/a-kansfield/job-journal>

B. Go to the GitHub front-end page for the Job Journal app and select “upload files”.



C. Drag and drop the contents of the browser folder in the provided space.



D. Confirm changes on the website URL (<https://a-kansfield.github.io/job-journal/>).

Section 4. Database

The application uses a PostgreSQL database attached to the back-end server through Heroku. Database information is available through the “Resources” tab of the app’s Heroku dashboard.

The screenshot shows the Heroku dashboard for the 'job-journal' application. The 'Resources' tab is selected, displaying the following information:

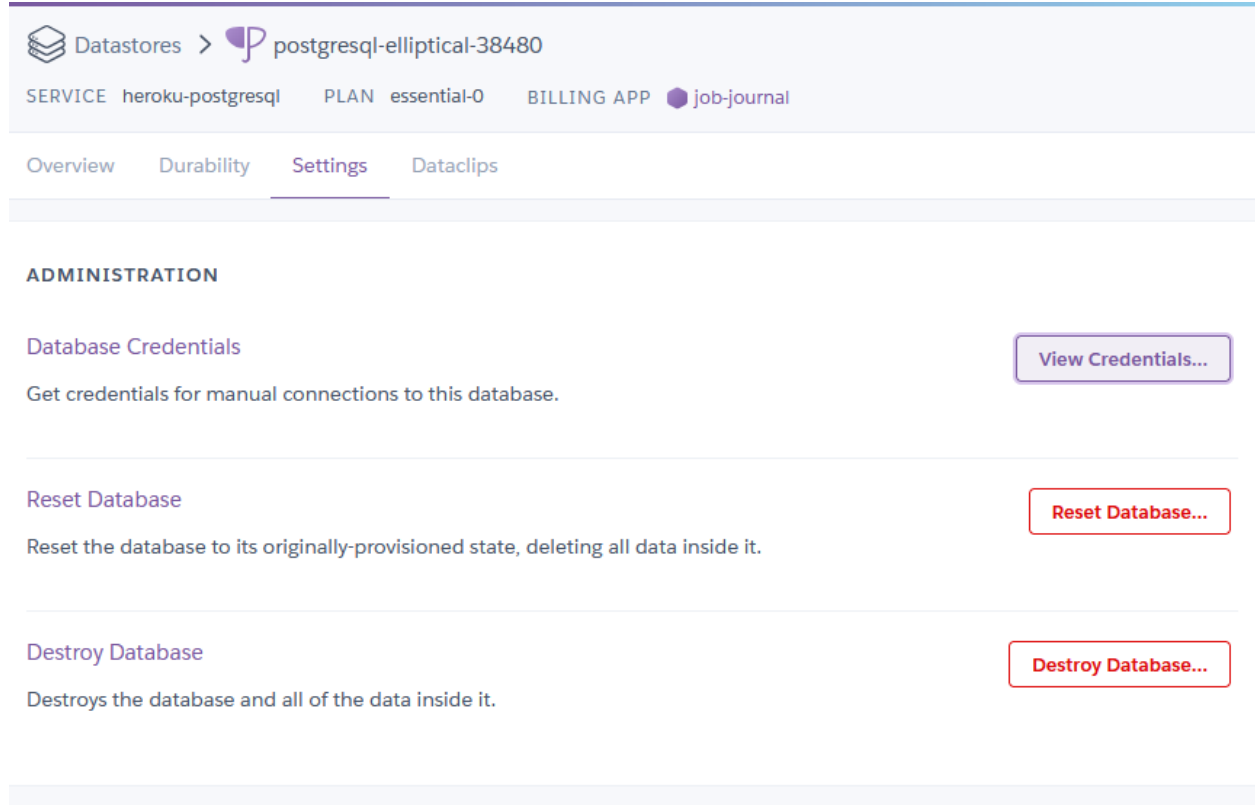
- Basic Dynos:** A single dyno named 'web' is running with the command `java -Dserver.port=$PORT $JAVA_OPTS -jar target/jobjournalcapstone-0.0.1-S...`. It is currently on and costs approximately \$0.010/hour.
- Add-on Services:** A search bar is present. Below it, a table lists the attached services.

Service	Attachments	Plan	Price	Actions
Heroku Postgres postgresql-elliptical-38480	Attached as DATABASE ↕	Essential 0	~\$0.007 / hour	⌵

Estimated Monthly Cost ~\$12.00/mo.

4.1 Connect to Database

If for any reason it is necessary to change the database schema, this can be accomplished on any database GUI or command line. The credentials to access the database can be found when selecting the button to the right of “Heroku Postgres.”



The screenshot shows the Heroku PostgreSQL settings page for a database named 'postgresql-elliptical-38480'. The breadcrumb trail is 'Datastores > postgresql-elliptical-38480'. Below this, there are links for 'SERVICE heroku-postgresql', 'PLAN essential-0', 'BILLING APP', and 'job-journal'. A navigation bar includes 'Overview', 'Durability', 'Settings' (which is highlighted), and 'Dataclips'. The 'ADMINISTRATION' section contains three items: 'Database Credentials' with a 'View Credentials...' button, 'Reset Database' with a 'Reset Database...' button, and 'Destroy Database' with a 'Destroy Database...' button. Each item has a brief description of its function.

ADMINISTRATION

Database Credentials [View Credentials...](#)
Get credentials for manual connections to this database.

Reset Database [Reset Database...](#)
Reset the database to its originally-provisioned state, deleting all data inside it.

Destroy Database [Destroy Database...](#)
Destroys the database and all of the data inside it.

Under “View Credentials,” you will find everything you need to connect to the PostgreSQL database.

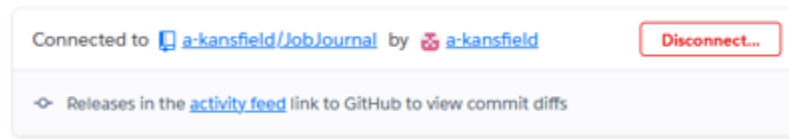
4.2 Deletion

Heroku also allows you to clear the data while keeping the schema intact or destroy the entire database.

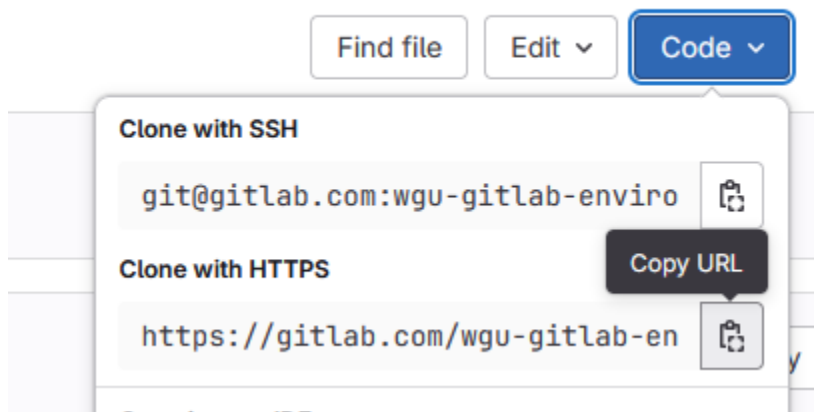
Section 5. GitHub Reset

With GitLab as the current git origin, there are certain restrictions that make pushing to GitHub as a second remote difficult and error-prone. It can sometimes be easier to bypass git troubleshooting entirely by re-importing the project when updates need to be pushed to Heroku. Note that this method will require downtime. The process for this is as follows:

- A. Disconnect the GitHub repository from Heroku.



- B. Delete the current GitHub JobJournal repository.
- C. Create a new repository (it is recommended to keep the name JobJournal)
- D. Select "Import repository."
- E. Use the https clone option in GitLab as shown below.



- F. Select "Begin Import."
- G. Wait for GitHub import to complete.
- H. Reconnect the newly created GitHub repository with your Heroku app. Follow Heroku instructions as normal from here.